

Competitive Programming

Section 28: DP I Mixed Contest

Competitive Programming Course

2026-07-10

Table of contents

1	Why a mixed DP contest is different	3
2	The DP diagnosis loop	4
3	Recognizing state shapes	5
4	Pattern 1: small previous-state DP	6
5	Pattern 2: two-dimensional knapsack	7
6	Pattern 3: tree knapsack	8
7	Postorder matters	9
8	Complexity	10
9	C++ notes	11
10	Python notes	12
11	Proof habits	13
12	Common mistakes	14
13	Submission checklist	15
14	Exercises	16

1 Why a mixed DP contest is different

In a topic lesson, the problem usually tells you what to practice. In a contest, the statement does not say:

```
This is 1D DP.  
This is knapsack.  
This is tree DP.  
This is bitmask DP.
```

You have to infer the state from constraints and from the shape of the choices. This section reviews the first DP block and adds one important combination: **tree knapsack**, where a tree DP state also tracks a budget.

2 The DP diagnosis loop

Before writing code, force the problem through this checklist:

1. What decisions are made one by one?
2. What information from the past affects the future?
3. Can that information be compressed into an index, mask, interval, node, or budget?
4. What is the base state?
5. Does each transition preserve the state meaning?
6. What exact state contains the final answer?

Most wrong DP submissions fail at step 2 or 6.

3 Recognizing state shapes

Statement signal	Likely DP state
“cannot repeat yesterday’s activity”	day and previous activity
“choose items within capacity”	used capacity
“two independent capacities”	used capacity in cart A and cart B
“if child chosen, parent must be chosen”	tree DP
“tree plus budget”	tree knapsack
“small set, visit/assign each once”	bitmask
“contiguous subarray/subsequence interval”	interval DP

4 Pattern 1: small previous-state DP

In the schedule problem, only yesterday's activity matters. We do not need to remember the whole week.

State:

```
dp[last] = minimum rest days after processed prefix,  
           where last is yesterday's action
```

Use:

```
last = 0  rest  
last = 1  C++ practice  
last = 2  Python practice
```

For each day:

```
new[0] = min(dp) + 1      # rest today  
new[1] = min(dp[0], dp[2]) # do C++ if available  
new[2] = min(dp[0], dp[1]) # do Python if available
```

Worked trace:

```
days = [3, 1, 2, 0]  
  
day 1: both available -> do either, 0 rests  
day 2: only C++       -> if yesterday was Python, do C++  
day 3: only Python    -> do Python  
day 4: none           -> rest
```

Minimum rests: 1.

5 Pattern 2: two-dimensional knapsack

Normal 0/1 knapsack tracks one capacity:

```
dp[w] = best value using weight w
```

With two carts, the remaining future depends on both used capacities:

```
dp[a][b] = best value with a weight used in cart A  
           and b weight used in cart B
```

For each item, there are three choices:

```
skip it  
put it in cart A  
put it in cart B
```

Transition:

```
if a + weight <= A:  
    dp[a + weight][b] = max(dp[a + weight][b], dp[a][b] + value)  
if b + weight <= B:  
    dp[a][b + weight] = max(dp[a][b + weight], dp[a][b] + value)
```

Because each item may be used at most once, iterate capacities in decreasing order:

```
for a from A down to 0:  
    for b from B down to 0:  
        ...
```

The decreasing order is the same idea as 0/1 knapsack. It prevents an item placed in one transition from becoming the source of another transition for the same item.

6 Pattern 3: tree knapsack

Tree knapsack combines two earlier ideas:

```
tree DP: solve child subtrees before parent
knapsack: track total budget
```

Prerequisite-tree statement:

```
If a lesson is selected, its parent must be selected.
```

For a node u , define:

```
dp[u][b] = maximum value inside subtree[u],
           using exactly b time,
           under the condition that u is selected
```

Why “ u is selected”? Because a child subtree can only contribute if the child is selected, and a child may be selected only when its parent is selected.

Initialization:

```
dp[u][time[u]] = value[u]
```

Then merge children one by one. For child v :

```
merged = dp[u]                # choosing nothing from child v is allowed
for used in 0..B:
    for add in 1..B-used:
        merged[used + add] =
            max(merged[used + add], dp[u][used] + dp[v][add])
dp[u] = merged
```

The $\text{add} = 0$ case is already represented by copying $\text{dp}[u]$ into merged .

Diagram:

```
    u
   / | \
  a  b  c
```

When merging child b :

$\text{old dp}[u]$ already includes choices from processed children.

$\text{dp}[b]$ summarizes every valid prerequisite-respecting choice inside b 's subtree.

7 Postorder matters

For tree knapsack, every child DP must be ready before its parent. Do not rely on node numbers unless the statement explicitly guarantees parent IDs are smaller than child IDs.

Build an order from the root:

```
order = [root]
for u in order:
    append children of u

process reversed(order)
```

8 Complexity

Problem type	Typical complexity
schedule with 3 previous states	$O(n)$
two-capacity knapsack	$O(n \cdot A \cdot B)$
tree knapsack	$O(n \cdot B^2)$

The tree knapsack bound comes from merging budget `used` with budget `add` for each child.

9 C++ notes

Use `long long` for values. A convenient negative infinity:

```
const long long NEG = -(1LL << 60);
```

For two-dimensional vectors:

```
vector<vector<long long>> dp(A + 1, vector<long long>(B + 1, 0));
```

For tree knapsack:

```
vector<vector<long long>> dp(n, vector<long long>(budget + 1, NEG));
```

10 Python notes

Python lists are enough for the local constraints. When merging a child in tree knapsack, copy the current row:

```
merged = dp[u][:]
```

Do not accidentally alias rows:

```
# wrong
dp = [[0] * (B + 1)] * n

# right
dp = [[0] * (B + 1) for _ in range(n)]
```

11 Proof habits

For mixed DP, the proof should match the state:

```
previous-state DP: induction over days  
knapsack: induction over processed items  
tree DP: induction over subtree height / postorder
```

Tree knapsack proof usually says:

```
Assume every child table is correct.  
Before merging a child, dp[u] is correct for already processed children.  
The merge either takes nothing from the child or combines a valid budget from  
u's processed part with a valid budget from the child.  
Therefore after all children are merged, dp[u] is correct.
```

12 Common mistakes

```
Keeping too much state and timing out.  
Keeping too little state and losing necessary information.  
Iterating 0/1 knapsack capacities upward.  
Forgetting the "skip child" case in tree knapsack.  
Allowing a child lesson without its parent.  
Assuming root must be selected when selecting nothing may be optimal.
```

13 Submission checklist

1. State meaning is written in one sentence.
2. Base cases are initialized exactly once.
3. Transition cases match all legal choices.
4. The final answer uses "at most" budget if the problem says at most.
5. Large values use 64-bit integers.
6. Random small tests can be brute-forced.

14 Exercises

For each task, first name the state dimension that prevents a greedy solution: day for A, both positions for B, and subtree dependency for C. Test the smallest feasible instance before timing the mixed set.